ТОО ”Колледж Хекслет”

Лабораторная работа №8

Миграции и начальное заполнение базы данных

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Миграции и начальное заполнение базы данных.

Цель: Научиться создавать и применять миграции Prisma, контролировать историю изменений схемы и воспроизводимо заполнять development database начальными данными.

Условие: RO4_3/lesson8/ЛР-8.pdf

Требования задания

• Поле description добавлено в Product через отдельную миграцию.

• Миграция хранится в prisma/migrations.

• Содержимое migration.sql изучено и соответствует изменению модели.

• prisma migrate status не показывает неприменённых миграций.

• После изменения схемы выполнен prisma generate.

• Product API работает с полем description.

• prisma.config.ts содержит команду seed.

• Файл prisma/seed.js создаёт стартовые категории и товары.

• Повторный seed не создаёт неконтролируемые дубликаты.

• Данные проверены через Prisma Studio.

• Самостоятельная миграция Category.slug выполнена отдельно.

Ход выполнения

Отдельная миграция добавляет nullable Product.description и сохраняет существующие строки. Самостоятельная миграция добавляет Category.slug с уникальным индексом. Seed использует upsert категорий и проверяет наличие товаров до create. Повторный запуск не создаёт дублей. migrate dev разрабатывает миграции, migrate deploy применяет сохранённые, generate обновляет клиент, seed добавляет данные.

Проект: RO4_3/lesson8

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

prisma.config.ts

```ts
import 'dotenv/config';
import { defineConfig, env } from 'prisma/config';
export default defineConfig({
    schema: 'prisma/schema.prisma',
    migrations: { path: 'prisma/migrations', seed: 'tsx prisma/seed.js' },
    datasource: { url: env('DATABASE_URL') }
});
```

Листинг 1. prisma.config.ts

prisma/migrations/20261104000200_add_product_description/migration.sql

```sql
ALTER TABLE "products" ADD COLUMN "description" TEXT;
```

Листинг 2. prisma/migrations/20261104000200_add_product_description/migration.sql

prisma/migrations/20261104000300_add_category_slug/migration.sql

```sql
ALTER TABLE "categories" ADD COLUMN "slug" TEXT;
CREATE UNIQUE INDEX "categories_slug_key" ON "categories"("slug");
```

Листинг 3. prisma/migrations/20261104000300_add_category_slug/migration.sql

prisma/seed.js

prisma/seed.js

```
import { prisma } from '../config/prisma.js';
async function main() {
    const laptops = await prisma.category.upsert({ where: { name: 'Ноутбуки' }, update: { slug: 'laptops' }, create: { name: 'Ноутбуки', slug: 'laptops' } });
    const accessories = await prisma.category.upsert({ where: { name: 'Аксессуары' }, update: { slug: 'accessories' }, create: { name: 'Аксессуары', slug: 'accessories' } });
    for (const data of [
        { name: 'Laptop Pro', price: 420000, description: 'Тестовый ноутбук', categoryId: laptops.id, inStock: true },
        { name: 'Wireless Mouse', price: 12000, description: 'Тестовая беспроводная мышь', categoryId: accessories.id, inStock: true }
    ]) {
        const existing = await prisma.product.findFirst({ where: { name: data.name, categoryId: data.categoryId } });
        if (!existing) await prisma.product.create({ data });
    }
    console.log('Seed completed');
}
try { await main(); } catch (error) { console.error(error.message); process.exitCode = 1; }
finally { await prisma.$disconnect(); }
```

Листинг 4. prisma/seed.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Миграции description/slug применены; повторный seed без дублей

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

POST /api/products → 201

DELETE /api/products/8 → 200

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "description": null,
  "categoryId": 1,
  "inStock": true
}
```

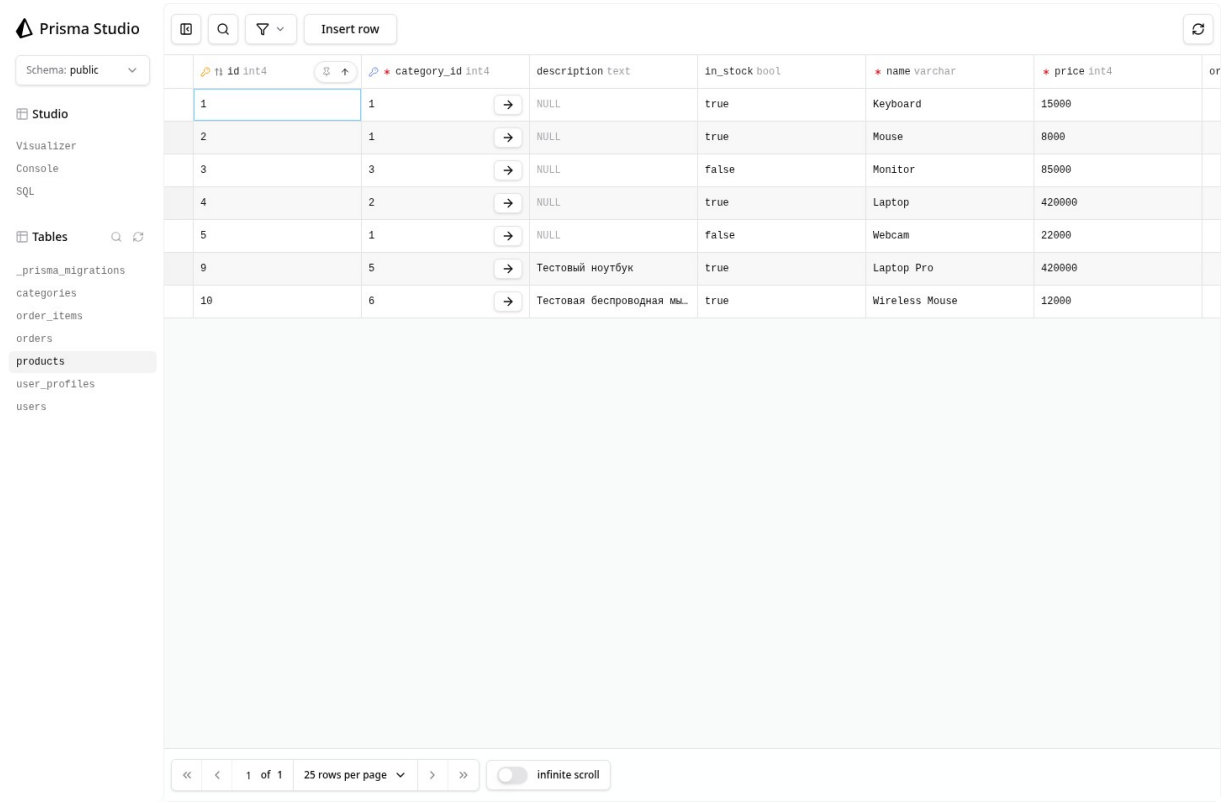

PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "description": null,
  "categoryId": 1,
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "description": null,
  "categoryId": 1,
  "inStock": false
}
```

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.